UCAM

Práctica 1

Laboratorio 1 - Git Fundamentals

Administración de Bases de Datos

Grado en Ingeniería Informática

https://github.com/csanchezz04/oracle-database-lab

Docente: Richard Avilés López

Curso académico: 2026/2027



Preguntas de comprobación

1. ¿Cuál es la diferencia entre Working Directory, Staging Area y Local Repository? Da un ejemplo de un archivo pasando por las tres.

- El Working Directory es la carpeta donde modificamos los archivos. La Staging Area contiene los cambios que hemos seleccionado con 'git add' para incluirlos en el próximo commit. El Local Repository almacena los commits del historial.
- Por ejemplo, modifico 'README.md', por lo que el cambio está en el Working Directory. Después ejecuto 'git add README.md' y pasa a la Staging Area. Finalmente, con 'git commit', queda registrado en el repositorio local.

2. Si modificas un archivo pero no haces git add, ¿aparece ese cambio en tu próximo commit? Explica por qué.

- No. Si modifico un archivo pero no ejecuto 'git add', ese cambio no se incluye en el próximo commit. Git registra los cambios que se encuentran preparados en la Staging Area.

3. ¿Por qué git status no mostraba las carpetas vacías que creaste en la Parte C? ¿Qué truco usamos para solucionarlo?

- Git no registra carpetas vacías, sino archivos. Por eso, las carpetas vacías que creamos no aparecían en el estado del repositorio. Utilizamos archivos '.gitkeep' para mantenerlas en el repositorio.

4. Explica con tus palabras qué es HEAD.

- HEAD es una referencia que indica el commit en el que estamos situados actualmente, normalmente a través de la rama activa. Al cambiar de rama, HEAD pasa a señalar la nueva posición.

5. ¿Qué diferencia hay entre crear una branch con git switch -c y crear una carpeta nueva con mkdir? ¿Cómo lo comprobamos en la Parte G?

- 'git switch -c' crea una rama nueva y cambia a ella, permitiendo trabajar en una línea de desarrollo independiente. 'mkdir' únicamente crea una carpeta en el sistema de archivos.
- En la Parte G lo comprobamos creando 'feature/customer-search'.

6. Durante el conflicto de la Parte H, ¿qué representaba el contenido entre <<<<<<< HEAD y =======? ¿Y entre ======= y >>>>>>>?

- El contenido entre '<<<<<<< HEAD' y '=======' representa nuestra versión actual. El contenido entre '=======' y '>>>>>>>' representa la versión de la rama que estamos intentando integrar.

7. ¿Por qué NO se debe hacer git commit --amend sobre un commit que ya se subió con git push?

- Porque modifica el commit anterior y genera un nuevo identificador.

8. Si borras por accidente la carpeta .git de tu proyecto, ¿qué se pierde exactamente? ¿Se pierde también el código fuente que está en el disco?

- Se pierde el historial de Git, los commits, las ramas, la configuración del repositorio y la información del remoto. Los archivos del proyecto que estén fuera de '.git' siguen en el disco

9. Explica con tus propias palabras la diferencia entre Git y GitHub, sin usar la palabra "nube".

- Git es un sistema de control de versiones que permite registrar cambios, crear ramas y consultar el historial de un proyecto. GitHub es una plataforma que aloja repositorios Git y facilita compartir código, colaborar y revisar cambios.

10. ¿Por qué no se debe subir un archivo .env con contraseñas reales a un repositorio, aunque el repositorio sea privado?

- Porque contiene información confidencial, como contraseñas, claves o tokens. Aunque el repositorio sea privado, las credenciales podrían quedar expuestas por errores de permisos, accesos no autorizados o futuras publicaciones.

11. Un compañero te dice: "hice push y ahora GitHub me rechaza el segundo push con 'non-fast-forward'". ¿Qué ha ocurrido probablemente y qué comando ejecutarías primero?

- Probablemente, la rama remota contiene commits que todavía no están en nuestra rama local, por ejemplo, porque alguien modificó el repositorio desde GitHub. Primero ejecutaría 'git status' para comprobar el estado local y después 'git pull' para recuperar e integrar los cambios, resolviendo posibles conflictos antes de volver a hacer push.

12. ¿Qué tipo de Conventional Commit (feat, fix, docs, test…) usarías para: añadir un índice de rendimiento a una tabla, corregir una restricción mal definida, y actualizar el README?

- Añadir un índice para mejorar el rendimiento de una tabla: 'perf: add performance index to table'.

- Corregir una restricción mal definida: 'fix: correct table constraint'.

- Actualizar el README: 'docs: update README'.